

Experiment No: 06

Machine Learning

Aim:

Optimize SVM hyperparameters (C, kernel) via grid search on a synthetic 2D dataset.

Objective:

- To understand the working of Support Vector Machines (SVM) for classification tasks.
- To learn how hyperparameters like C and kernel affect SVM model performance.
- To implement hyperparameter optimization using GridSearchCV.
- To analyze confusion matrix and classification report for performance evaluation.

Prerequisites:

- Software: Python 3.x with libraries: pandas, scikit-learn, numpy.
- Hardware: A system with at least 4 GB RAM.
- Skills: Basic Python programming and understanding of SVMs and model evaluation.

Theory:

Support Vector Machines (SVM) are powerful supervised learning models used for classification and regression tasks. They work by finding the optimal hyperplane that separates data points of different classes with the maximum margin.

The performance of an SVM largely depends on its hyperparameters:

- The regularization parameter (C) controls the trade-off between maximizing the margin and minimizing classification errors.
- The kernel function (linear, polynomial, RBF, etc.) maps the input data into higher dimensions, making it possible to handle non-linear relationships.

Hyperparameter tuning is crucial for achieving optimal model performance. Grid Search systematically explores a predefined set of hyperparameter combinations and selects the one that yields the best cross-validation score.

Apparatus/Software Requirements:

- Python IDE (Jupyter Notebook, VS Code, etc.).
- Required Python libraries: pandas, scikit-learn, numpy.
- A synthetic 2D dataset for visualization and model training.

Procedure:

Step 1: Load or generate a synthetic 2D dataset and inspect its structure.

Step 2: Preprocess the data (scaling and encoding if required).

Step 3: Split the dataset into training and testing sets.

Step 4: Define an SVM model and parameter grid for C, kernel, and gamma values.

Step 5: Perform GridSearchCV to identify the best hyperparameter combination.

Step 6: Evaluate the best model using the test dataset.

Step 7: Display confusion matrix and classification report for model performance.

Program Code:

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.svm import SVC
from sklearn.metrics import classification_report, confusion_matrix
```

1. Load dataset

```
df = pd.read_csv("synthetic_dataset.csv")
print("Dataset Shape:", df.shape)
print(df.head())
```

2. Define features and target

```
X = df.drop(columns=["target"], errors="ignore")
y = df["target"]
```

3. Encode target if necessary

```
le = LabelEncoder()
y = le.fit_transform(y)
```

4. Train-test split

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
```

5. Scale features

```
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

6. SVM + GridSearchCV

```
param_grid = {
    "C": [0.1, 1, 10],
    "kernel": ["linear", "rbf", "poly"],
    "gamma": ["scale", "auto"]
}
svm = SVC()
grid = GridSearchCV(svm, param_grid, cv=3, scoring="accuracy", n_jobs=-1)
grid.fit(X_train, y_train)
```

7. Results

```
print("Best Parameters:", grid.best_params_)
print("Best CV Accuracy:", grid.best_score_)
```

8. Test set evaluation

```
best_model = grid.best_estimator_
y_pred = best_model.predict(X_test)
print("\nConfusion Matrix:\n", confusion_matrix(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test, y_pred))
```

Expected Output:

- Display dataset shape and first few rows.

```
Dataset Shape: (569, 32)
   id diagnosis  radius_mean  texture_mean  perimeter_mean  area_mean  \
0   842302      M        17.99        10.38         122.80       1001.0
1   842517      M        20.57        17.77         132.90       1326.0
2   84300903     M        19.69        21.25         130.00       1203.0
3   84348301     M        11.42        20.38          77.58        386.1
4   84358402     M        20.29        14.34         135.10       1297.0

   smoothness_mean  compactness_mean  concavity_mean  concave_points_mean  \
0         0.11840         0.27760         0.3001         0.14710
1         0.08474         0.07864         0.0869         0.07017
2         0.10960         0.15990         0.1974         0.12790
3         0.14250         0.28390         0.2414         0.10520
4         0.10030         0.13280         0.1980         0.10430

   ... radius_worst  texture_worst  perimeter_worst  area_worst  \
0   ...         25.38         17.33         184.60        2019.0
1   ...         24.99         23.41         158.80        1956.0
2   ...         23.57         25.53         152.50        1709.0
3   ...         14.91         26.50          98.87         567.7
4   ...         22.54         16.67         152.20        1575.0
```

- Best hyperparameters and corresponding cross-validation accuracy.

```
Best Parameters: {'C': 0.1, 'gamma': 'scale', 'kernel': 'linear'}  
Best CV Accuracy: 0.9780411293133495
```

- Confusion matrix and classification report.

```
Confusion Matrix:  
[[72  0]  
 [ 2 40]]  
  
Classification Report:  
              precision    recall  f1-score   support  
  
     0       0.97         1.00         0.99         72  
     1       1.00         0.95         0.98         42  
  
 accuracy          0.98         0.98         0.98        114  
 macro avg          0.99         0.98         0.98        114  
 weighted avg          0.98         0.98         0.98        114
```

Observations:

The optimized SVM model shows improved accuracy after hyperparameter tuning. Grid Search helps systematically identify the best parameter set for optimal model performance.

Conclusion:

This experiment demonstrated how hyperparameter optimization using GridSearchCV enhances the performance of Support Vector Machines. The process provided insights into the influence of C, kernel, and gamma on model generalization.

Viva Questions:

- What does the parameter C control in SVM?
- What is the role of kernel functions in SVM?
- Why is feature scaling important before training an SVM?
- What is the purpose of GridSearchCV?
- How can we handle overfitting in SVM models?